

dsh 图像管线深度解析

图片是如何进入大模型的 —— 存储投影传输全链路

面向 AI 应用开发者

deepseek-harness v0.1.1-rc.2

四方对比: dsh · opencode · Codex · Claude Code

目录

1. 基础: 模型到底怎么"看"图
2. 为什么裸发 base64 会出事
3. dsh 全景架构
4. 逐层拆解 (规范化 / 投影缓存 / Files API / 预算闸门 / 坐标换算 / 历史治理)
5. 四方方案对比: dsh vs opencode vs Codex vs Claude Code
6. 实践建议
7. 术语速查

一、基础: 模型到底怎么"看"图

先建立最重要的认知: **大模型本身不认识 PNG/JPEG 文件格式, 它只认识数字矩阵。**

当你说"把这张图发给 GPT", 实际发生的流程是:

你的 PNG 文件 (二进制字节)

| 解码



像素矩阵 [R,G,B] [R,G,B] ... → 模型内部的视觉编码器 (ViT 等)

| 把像素块切成 patch、转成 token



模型"看到"的是一串特殊的图像 token

而**传输协议**层面, 主流只有三张"面孔", 本质都是同一张图的不同包装:

形态	样子	用在哪家
base64 data URL	<code>data:image/png;base64,iVBORw0KGgo...</code>	OpenAI Chat Completions、Claude Messages、DeepSeek、Gemini —— 几乎所有家
file_id 引用	<code>{ "type": "file", "file_id": "file-abc123" }</code>	DeepSeek Files API、OpenAI Files
公网 URL	<code>{ "type": "image_url", "image_url": { "url": "https://..." } }</code>	OpenAI（服务端代抓取）、Gemini、Claude（URL source）

所以回答一个常见疑问：**是的，模型侧最终收到的仍然是 base64 编码的像素**（或等价的 file_id 由服务商自己解码）。所谓“图像管线”，全部工作都发生在**你的客户端把文件字节变成请求体里那串字符之前**——这一段做得好不好，直接决定了成本、速度和可靠性。

1.1 图片是怎么被模型“计费”的

以视觉 token 的通用计算方式为例：

- **OpenAI 系**：图片按 patch 计费。`high` detail 下，一张图先缩放到长边 $\leq 2048\text{px}$ ，再按 512px 正方形切 patch，每个 patch 约 170 token，外加 85 token 基础费；`low` detail 固定 85 token。
- **Anthropic 系**： $\text{token} \approx (\text{宽} \times \text{高}) \div 750$ 。
- **DeepSeek 系**：按归一化后的像素预算折算。

关键结论：图片尺寸直接等于钱和延迟。一张 4K 截图不压缩就发，可能一次就是数千 token；而人眼识别内容往往只需要 1024px。这就是所有智能体都要做降采样的根本原因——**不是怕请求太大，是怕 token 太贵。**

二、为什么裸发 base64 会出事

假设你写了个最朴素的循环对话脚本：每次都把历史消息里的所有图片原样重发。用不了多久你会撞上四堵墙：

墙 1：体积膨胀

base64 编码把 3 字节变 4 字符，净膨胀 33%。一张手机照片 4MB，编码后 5.3MB 文本；一次贴 10 张图，请求体 50MB+。很多网关/代理默认拒绝超过 1~10MB 的请求体。

墙 2: KV-cache 反复失效

这是最隐蔽也最贵的一堵墙。主流 provider 都做 **prefix caching**: 如果本次请求的前 N 个 token 和上次完全一样, 这部分不用重新计算, 费用打一折、延迟大降。但“完全一样”是**字节级**判定。如果你每次对原图重新压缩编码, 压缩器的输出哪怕差一个字节, 从那张图开始往后的所有缓存全部作废。多轮对话里反复引用同一张截图时, 朴素做法等于每轮都在全价重算。

墙 3: 无法重放

会话日志如果把 base64 全存下来, 一天的开发会话日志能到 GB 级。想回看“上周那张架构图”? 翻不动。想在 fork 出的新分支里继续引用旧图? 字节已经不存在了。

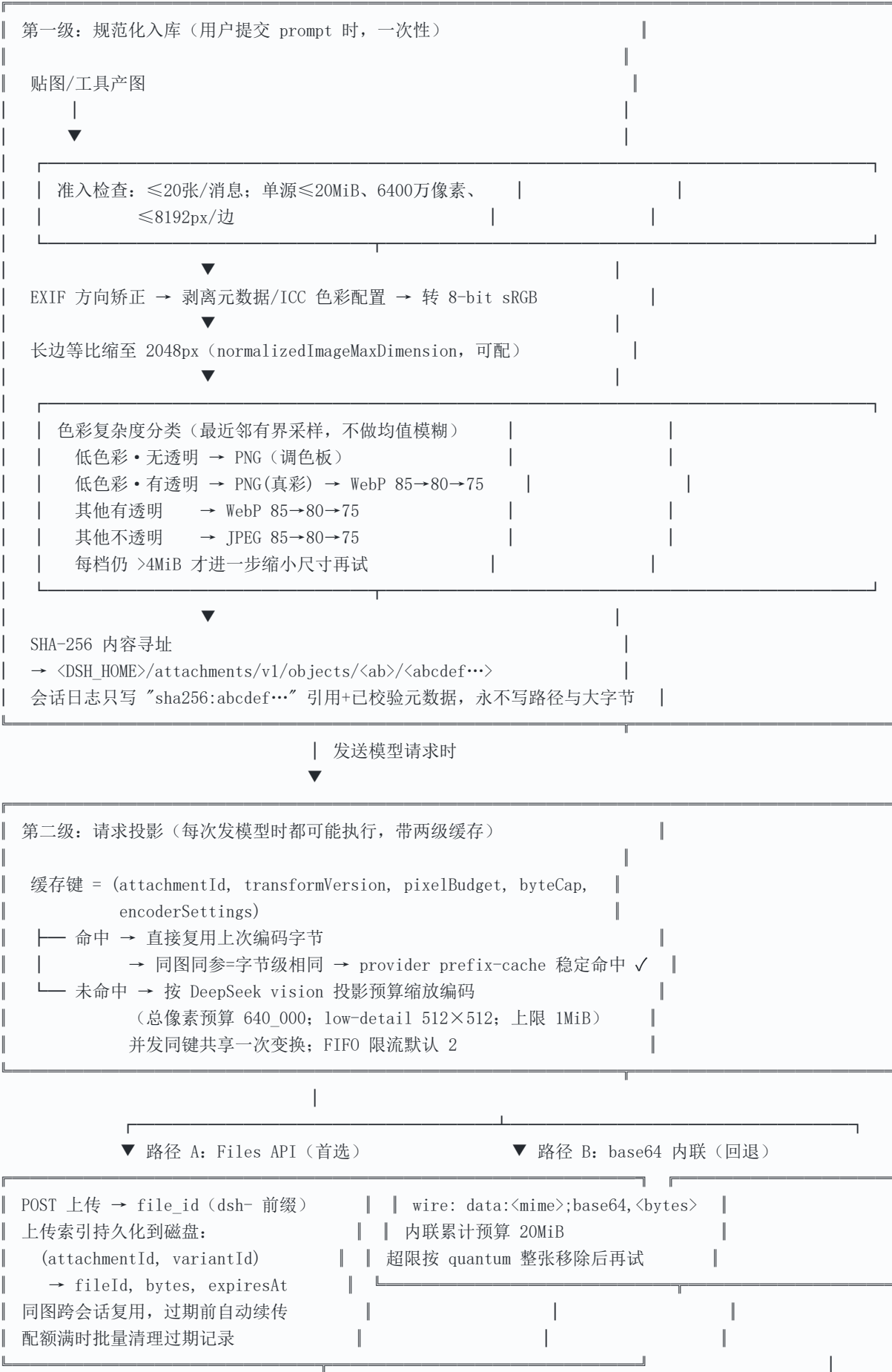
墙 4: 无预算控制

用户一次拖进来 30 张 8K 屏幕截图, 或者某个工具每轮返回一张高清渲染图。没有闸门的话, 要么请求被 provider 拒绝 (Claude 对 >20 张图的请求有更严格的单图尺寸限制), 要么账单爆炸。

dsh 的图像管线就是为了系统性地拆掉这四堵墙。核心思路一句话: 图片存一次 (内容寻址), 发送时按需投影 (带缓存), 传输走引用优先 (file_id), 全链路有预算闸门。

三、dsh 全景架构

dsh 在 **0.1.1-rc.2** 中把图片处理重构为一条三级流水线。先看全景, 再逐级拆解:



第三级：wire 格式（DeepSeek 协议不变）+ 预算闸门

{ type:"file", file_id } 或

{ type:"image_url", image_url:{ url:"data:...;base64,..." } }

闸门：Files 引用累计 $\leq 128\text{MiB}$ / 内联累计 $\leq 20\text{MiB}$ /

单图编码 $\leq 1\text{MiB}$ / 单请求 ≤ 600 张

超限按 quantum 步长整张剔除（确定性：同样输入永远剔除同一批）

源码锚点：第一级 `packages/attachment/attachment-local/src/index.ts`；第二级 `request-images.ts`；第三级 `llm-deepseek/src/{serialize,file-store,upload-index}.ts`。

四、逐层拆解

4.1 规范化入库：一张图只认真处理一次

这一级的核心理念是**内容寻址（Content-Addressed Storage）**：数据的哈希值就是它的地址。改一个像素，哈希全变——天然去重、天然防篡改。

准入即拒绝

每条消息最多 20 张图、累计 200MiB 源字节；单张最大 20MiB、6400 万像素、单边 8192px。超限直接报错，不会默默吞掉。

清洗三连

1. **EXIF 方向矫正**——手机拍照常带旋转标记，不矫正的话模型看到的是横躺的图；
2. **剥除元数据与 ICC 色彩配置**——GPS 坐标、拍摄参数对模型无用还占体积，且避免隐私泄露；
3. **统一 8-bit sRGB/sRGBA**——16-bit 深度、CMYK 等异构色彩空间强制转换，保证后续每一步的字节可重现。

色彩复杂度感知压缩

压缩前先用**最近邻有界采样**判断图片是"低色彩"（截图、图表、二维码）还是"高色彩"（照片）：

- 低色彩图走 PNG 路线——终端截图常常能压到几十 KB；
- 有透明通道必须保 alpha（WebP/PNG），绝不能走 JPEG（会变黑底）；
- 照片走 JPEG 逐档降质（85→80→75）。

每一档质量不行才降下一档，**尺寸缩减只在所有质量档都失败后才进行**——优先保清晰度，其次保体积。

直通优化 (passthrough)

如果原图已经是干净的 8-bit sRGB PNG/JPEG/WebP 且在限制内，**逐字节原样通过**，完全不重编码——零损失零开销的快路径。GIF 动图只取首帧（v1 契约明确排除动画，诚实声明而非静默劣化）。

入库之后

最终字节算 SHA-256 存为对象。会话日志里这张图只剩一行引用 `sha256:e3b0c44298fc...` 加已验证元数据（宽高、媒体类型）。**日志从此与图片体积解耦**。

4.2 请求投影：保护 KV-cache 的核心设计

这是整个管线最有价值的一级。

问题回顾：provider 的 prefix caching 要求请求前缀字节级相同才命中。传统做法每次对原图重新跑一遍压缩，压缩器版本升级、参数微调、甚至并发时序差异都会导致输出字节漂移——前缀从那张图开始全部作废，整段重算。

dsh 的解法：把“图片编码”改造成确定性纯函数，并给结果加缓存。

```
缓存键 = (attachmentId, transformVersion, pixelBudget, byteCap, encoderSettings)
```

五个字段缺一不可：

- `attachmentId`：哪张图（sha256）
- `transformVersion`：变换算法版本——管线升级时递增，旧缓存自然淘汰而不是错误命中
- `pixelBudget / byteCap`：本次请求的预算（DeepSeek vision 默认总像素 640_000，编码上限 1MiB）
- `encoderSettings`：编码器固定参数

同键必同字节，于是：

1. 多轮对话反复引用同一张截图 → 除首轮外，每轮该图的字节**完全一致**
2. provider 的 prefix cache 从头到尾稳定命中 → 长会话成本大幅下降
3. 缩放只在“不放大”前提下进行（小图保持原样），避免无意义放大浪费 token

工程细节：并发展开时多个协程同时要同一张图的投影，共享同一次变换（一个算，其他等）；全局 FIFO 限流器控制同时进行的压缩数量（`imageCompressionConcurrency`，1~8，默认 2）。取消其中一个等待者不会取消共享的工作本身。缓存的字节在复用前还会被完整解码校验一次（确保确实是合法 8-bit sRGB），防止缓存被污染。

4.3 Files API 与 file_id：请求体从 30MB 到几 KB

DeepSeek 提供文件上传服务：先把图 POST 到 Files 端点拿一个 `file_id`，之后聊天请求体里只放这个短字符串，服务端自己去关联存储取图。

rc.2 把这条路径做成**首选**，并补齐了让它生产可用的全部配套：

① 持久化上传索引 (upload-index.ts)

磁盘上的映射表（带 `formatVersion: 3`、原子写 + 文件锁防多进程竞争）：

```
interface DeepSeekUploadRecord {
  scope: DeepSeekFileScope      // 按 apiKey 派生的非秘密命名空间（密钥本身不落盘）
  attachmentId: AttachmentId    // 哪张规范化图
  variantId: ImageVariantIdType // 哪种投影变体（含路由预算）
  fileId: DeepSeekFileIdType     // 远端 file-xxx
  bytes: number
  createdAt / expiresAt         // 过期时间
}
```

效果：同一张图跨会话、跨重启都不再重复上传。第二次用到时查索引直接复用还没过期的 `file_id`；临近过期则按 `refreshMarginSeconds` 自动重传续期。索引的命名空间从 `apiKey` 哈希派生——不同 key 的上传互不污染，且密钥本身永不落盘或进日志。

② 并发去重与取消传播

多个请求同时要传同一张图时，合并为一次上传操作（SharedUpload 模式），大家等同一个 Promise；若所有等待者都取消了且上传尚未完成，才真正 abort 底层请求——避免“取消了一个等待者就把共享上传搞挂”的误伤。

③ 失败回退

Files API 超时（`filesApiTimeoutMs`，rc.2 专门把它与聊天流式超时解耦）或配额错误时，自动降级为路径 B base64 内联。功能不断，只是那次请求变大。

④ 配额回收

上传空间有配额，满了按批次清理已过期的记录再试。

澄清常见误解：file_id 不是给模型“自己去索引图片”用的。模型看到的只是协议字段，取图发生在 DeepSeek 服务端。它的收益纯粹在网络与成本层——请求体从几十 MB 缩到几 KB，且上传一次到处引用。

4.4 预算闸门：确定性剔除，而非分批

层	默认值	保护对象
Files 引用累计	128 MiB	单请求中所有 file_id 代表的原始字节
base64 内联回退累计	20 MiB	回退后的请求体大小
单图投影编码	1 MiB	每张投到请求里的图
单请求图片数	600 张	极端防滥用

超限时按 `quantum` 步长**整张移除图片**——不是把图切片分批发，而是这张图这个请求不带（文本部分照发，图的位置留占位说明）。

为什么强调确定性：剔除逻辑必须满足“同样输入永远剔同样几张”。这样今天被剔的图明天重放会话时还是被剔，日志、实际请求、计费三者永远对得上。随机或依赖时序的剔除策略会让重放失真。

另有一层**上下文溢出联动**：当 provider 返回“上下文窗口超限”错误时，管线绕过常规压力检查，先尝试剔除大图再重试——因为 provider 已经确认了“确实塞不下”，此时无需容量元数据即可行动。

4.5 read_image 坐标换算：视觉工具的地基

rc.2 给 `tool-fs read_image` 工具加了两个返回值：**缩略图的实际尺寸**和**相对原图的坐标比例尺**。

原因：发给模型的永远是按预算缩过的缩略图（比如 640k 像素），但模型经常需要“点击原图某个位置”或“裁剪原图某块区域”。有了比例尺，模型在缩略图上说的“(120, 80)”就能精确换算回原图坐标。这让 GUI 自动化、UI 标注、区域截取类插件（比如 hub 里收录的 modlens）有了标准化地基，不必各自实现一套换算。

配套地，attachment-local 实现了**确定性规范编码**：同一张图的规范化结果永远一致（版本化的转换算法 + 锁定的编码器行为）。这是一切缓存正确性的根基。

4.6 历史图片的上下文管理：老图怎么办

会话越滚越长，早期轮次里的图片占着大量 token。dsh 的做法：

- 投影缓存让“重发同一张图”成本接近零（字节相同，prefix cache 命中）
- 整体逼近上下文上限时，compaction 先于文本处理图片：可选的 `toolResultPruner` 先改写过大的工具结果，摘要再替换掉最老的历史区间
- 单张不可分割的图如果比保留预算还大，明确拒绝压缩并如实上报，而非静默丢图

五、四方方案对比：dsh vs opencode vs Codex vs Claude Code

我们逐一研读了四家的实现：dsh 本仓源码；opencode `packages/opencode/src/image/image.ts`（photon WASM 方案）；OpenAI Codex `codex-rs/core/src/image_preparation.rs` + `codex-rs/utils/image/src/lib.rs` + `compact_remote_v2_images.rs`（Rust 实现）；Claude Code 官方 Vision 文档与其 Read 工具行为。

5.1 先说清楚各家面对的协议约束

	传输形态	服务端缓存
DeepSeek (dsh 主目标)	base64 data URL 或 Files API <code>file_id</code>	prefix caching（自动）
OpenAI (Codex 目标)	Responses API: base64 data URL (<code>input_image</code>)；不支持远程 URL 直发	automatic prefix caching
Anthropic (Claude Code 目标)	Messages API: base64 <code>source</code> （也支持 URL source）	prompt caching（需显式 <code>cache_control</code> 标记）

三家都没有“客户端长期对象存储”的概念——图要么每次随请求重发，要么靠 provider 自己的历史管理。这就是各客户端图像管线的分水岭：你替 provider 管多少？

5.2 尺寸与压缩策略对比

维度	dsh	opencode	Codex	Claude Code
最大边	入库 2048px（可配）	2000×2000	high-detail 2048px; unified 模式 6000px	建议长边 ≤1568px; >20 张/请求时限 2000px
缩放算法	sharp/libvips (Lanczos 类)	photon WASM Lanczos3	Rust image crate Triangle 滤镜	客户端自理（官方 给缩放公式）
格式分支	按色彩复杂度选 PNG/WebP/JPEG 三分 支	先 PNG 再 JPEG 质 量 80/85/70/55/40	统一编码器	不做客户端转换 (原样发)
EXIF/ 色彩管 理	✅ 矫正+剥离+sRGB 强制	❌ 未显式处理	✅ 处理	❌ 客户端自理
动图	明确排除（取首帧）	未显式处理	未显式处理	支持有限动图

5.3 存储与重放对比

维度	dsh	opencode	Codex	Claude Code
持久对象 库	✅ sha256 内容寻址, <DSH_HOME>/attachments/	❌ 无, data URL 存于消 息	❌ 无独立库 (rollout 文件 记录原始项)	❌ 无 (会话 jsonl 记 base64)
重 启/fork 后可用	✅ 从对象库重建	⚠️ 取决于会 话文件留存	⚠️ rollout 含 原始数据	⚠️ 同左
日志体积	只存引用, 与图片体积解耦	含完整 base64	含原始数据	含完整 base64

5.4 缓存体系对比

维度	dsh	opencode	Codex	Claude Code
客户端编码缓存	✅ 两级：本地磁盘持久投影缓存 + 远端上传索引	❌ 每次 normalize 重算	✅ 进程内 LRU (32 条 / 64MiB, sha1 键)	❌ 无
跨会话复用	✅ 上传索引持久化，同图跨会话零重传	❌	⚠️ 仅进程生命周期内	❌
字节级确定性	✅ 设计目标 (transformVersion 版本化)	❌ 输出可能漂移	✅ 同版本下确定	— (不转换)

5.5 历史上下文里的图片治理（差异最大的维度）

策略	dsh	opencode	Codex	Claude Code
老图处理	compaction 时 pruner 改写大结果，摘要替换老区间；不可分割大图明确拒绝并上报	无专门机制（历史全量重发）	strip_image_details : lite 模型路由时剥掉历史图的 detail 字段； token 预算截断 : 裁历史时保证"图+相邻标签"原子保留或整体丢弃	依赖开发者手动管理（官方建议自行删旧图块或用 prompt caching）
超限行为	quantum 整张剔除（确定性）	迭代缩放 32 次后报错	占位符替换 ("image content omitted because...")	API 直接拒绝
给模型的反馈	剔除可见于日志	报错终止	ImageResizeNotice : 以 <code>images.resize_notice</code> 片段告知模型"第 N/M 张已从 WxH 缩至 W'xH'"	无

Codex 有几个值得单独点赞的设计：

- **ImageResizeNotice**——缩图后显式告知模型“你的图被缩了，原始多大、现在多大”。模型因此知道自己在看缩略图，坐标判断更谨慎。
- **detail 分级**——支持 high/low/auto/original 多档，lite 模型自动剥离历史图的 detail 以省 token。
- **patch 预算**——不只限尺寸还限 patch 总数（high 2500 / unified 10000），直接对应 OpenAI 计费粒度。

5.6 传输路径对比

维度	dsh	opencode	Codex	Claude Code
传输形态	file_id 优先 + base64 回退 双路径	仅 base64 内联	仅 base64 内联 (Responses API)	base64 为主（亦支持 URL source）
上传去重	✅ 持久索引，跨会话复用	—	—	—
失败回退	✅ Files 失败自动降级内联	—	—	—

5.7 总结：四种哲学

	一句话哲学	适用场景
dsh	"图片是一等公民资产：存一次、投影缓存、引用传输、全程可审计"	生产级多模态长会话、成本敏感、需审计重放的团队部署
opencode	"发送前把图弄到能塞为止"——轻量单点归一化	个人 CLI 使用、单轮单图、快速上手
Codex	"管好模型眼前的上下文"——精细的 resize 通知、detail 分级、patch 预算、历史 token 化裁剪	专业编码场景，图片多为工具产出，重视模型对自身输入的元认知
Claude Code	"协议原生，简单直接"——base64 直发 + 依靠 Anthropic API 的限制报错与 prompt caching	已在 Anthropic 生态内，图量可控

有趣的观察：dsh 和 Codex 各自做到了一半的“完备”。dsh 强在存储层（内容寻址 + 双级缓存 + 跨会话复用），Codex 强在上下文层（resize 通知 + detail 分级 + 原子截断）。理论上二者结合才是终极形态；实际上 dsh 的 read_image 坐标比例尺已经在朝 Codex 的 ImageResizeNotice 方向演进——都是让模型知道自己看到的是什么的图。

六、实践建议

给插件开发者

1. 不要自己压图。用 `ctx.attachments.saveImages()` 入库拿 `sha256:` 引用，编码/缓存/预算全部由管线承担。
2. 做视觉工具时务必利用 `read_image` 返回的坐标比例尺做精确换算，不要假设模型看到的尺寸 = 原图尺寸。
3. 工具返回大图前先想想能不能先缩——虽然管线兜底，省下来的就是用户的钱。

给应用架构师

1. 如果产品有多模态历史需求，dsh 的三层模式（内容寻址存储 → 确定性投影缓存 → 引用优先传输）值得整体借鉴，是目前开源智能体里最完整的实现。
2. 无论哪家 API，**保持投影参数恒定**是 prefix-cache 命中率的生命线——预算参数一旦变动，所有历史图片的缓存键全部变化，前缀全灭。

给成本敏感场景

1. 确认 provider 开启了 prefix caching（DeepSeek/OpenAI 自动；Anthropic 需显式 `cache_control`）。
2. 控制单图信息密度：截图尽量裁掉无关边缘；低色彩图（代码截图、终端输出）天然适合 PNG 路线，体积常只有照片的十分之一。
3. 长会话定期主动 compaction，别等被动溢出——被动恢复的代价是整段重算。

七、术语速查

术语	含义
base64 内联	图片二进制经 base64 编码为文本嵌入 JSON body。通用但膨胀 33%
file_id / Files API	图片先上传到服务商文件服务，请求携带短引用 ID，服务端自行取图
KV-cache / prefix caching	服务端缓存请求前缀的计算结果；前缀 字节级相同 才命中，否则从头重算
内容寻址存储	以数据内容的哈希值（SHA-256）作为存储地址，天然去重、写入不可变
确定性投影	相同输入必产生相同输出的编码过程；缓存正确性与重放一致性的前提
transformVersion	变换算法版本号；升级时递增使旧缓存自然淘汰而非错误命中
quantum 剔除	超预算时按固定步长整张移除图片（非切片），保证重放一致性
patch / 视觉 token	模型把图片切分成的小方块，每个 patch 折算固定数量的计费 token
detail 分级	OpenAI 特有：high（高分辨率多 patch） / low（固定 512×512 低耗） / auto
EXIF 方向	手机照片元数据里的旋转标记；不矫正会导致模型看到横躺的图
passthrough	已符合规范的图逐字节原样通过，跳过重编码的快路径
LruCache	最近最少使用缓存；Codex 用它做进程内图片编码缓存（32 条/64MiB）
rollout 文件	Codex 的会话记录格式，包含原始消息项（含图片数据）

基于 deepseek-harness v0.1.1-rc.2 (b150a55)、opencode (photon WASM 实现)、openai/codex (Rust core)、Anthropic Vision 官方文档源码/资料撰写 · 2026-08-24 · dsh-hub 项目
系列下一篇预告：《agent loop 的内部构造》